

Designing and building a GPS alarm

- ☒ ~~Choosing the problem and target users~~
- ☒ ~~Feature prioritization and scope~~
- ☒ ~~User flows and wireframes~~
- ☒ ~~UI design and visual style~~
- ☒ ~~Tech stack and architecture choices~~
- ☐ Implementation...

1. Choosing the problem and target users

Concept: An alarm clock

Let's say you are traveling a long distance. Long enough for you to doze off. Sometimes we wish we had an alarm clock that rings when you approach your destination. Not after some generic time window.

Therefore, now we build an alarm app that uses GPS location to wake you up, rather than a time clock.

This solves several problems:

- Maximise rest during journey
- Ensure no stress at the time of leaving the vehicle

- Account for delays/changes in reaching destination
 - Due to factors such as traffic, weather, etc.
 - Won't be addressed by a normal alarm clock
- Allows users to prepare for leaving - luggage, devices, etc.

Target Users:

General public. No subscription, no information stored in the cloud. No login tech.
All actions performed locally.

Use-case wise: Best for travellers or people in long journeys.

2.Feature prioritisation and Scope

The app will run fully locally with no data stored online (*Except maybe information necessary for advertisements*)

No login screens

Requires GPS permission. The app will not run without GPS.

Background battery usage permission mandatory.

User is able to adjust all aspects of the application - radius, GPS collection frequency, battery percentage cutoff, etc.

Considerations:

1. Battery (Long journeys, therefore battery should be considered)
2. GPS signal loss handling (Common in long journeys)
3. Estimations using suitable algorithm
4. Persistent memory for current trip / trip history
5. Alert user in case of any mishap (battery low, GPS loss, or app crash)
6. Running the app without internet connection

3. User flows and wireframes

User opens app:

First boot:

- Request GPS permission
- Request background usage permission
- Notification permission
- Display over other apps (for the alarm)
- Bypass silent mode and DND
- Permission to turn GPS on/off

If GPS or background usage permission are not given, app prompts for permission again. App closes.

If any of the other permissions is denied, produce warnings and continue operating

Subsequent boots:

Home screen:

Contains journey setup screen.

User may pick **Destination, alarm radius, Max/Min GPS polling frequency, alarm tune, low battery cutoff**

Also contains past journey in a list of cards.

If a journey is active, show that on top of the list.

TODO (resolved below): Revisit above for cases of GPS loss and reaching battery cutoff. Also consider the case where the current battery percentage is already below cutoff.

Journey setup:

The user picks a destination using an in-app map. Allows both searching and pinning location on map.

Download map cache to allow user to run app even without internet connection.

Visually show a circle of the chosen alarm radius. Adjust as radius is adjusted.

Radius can only be a maximum of 40% of the distance between user and the destination.

An advertisement is shown when a journey starts (**TODO:set up advertisements after app is ready**)

TODO (resolved below): Current UI design will not show anything about the Map-GPS component we might be using. So consider that later.

(Default) Settings screen:

Holds default values for all the above adjustable values.

For each journey that the user sets up, all values are adjustable. Starting values will be stored in this default settings screen.

About and donations screen:

Note from author,

Link for donations

CURRENT JOURNEY SCREEN:

Shows all selected values that the journey started with, including a map showing current location, alarm radius circle, and destination.

Consider showing current travel speed, estimated arrival, estimated alarm time, etc.

Background behaviour:

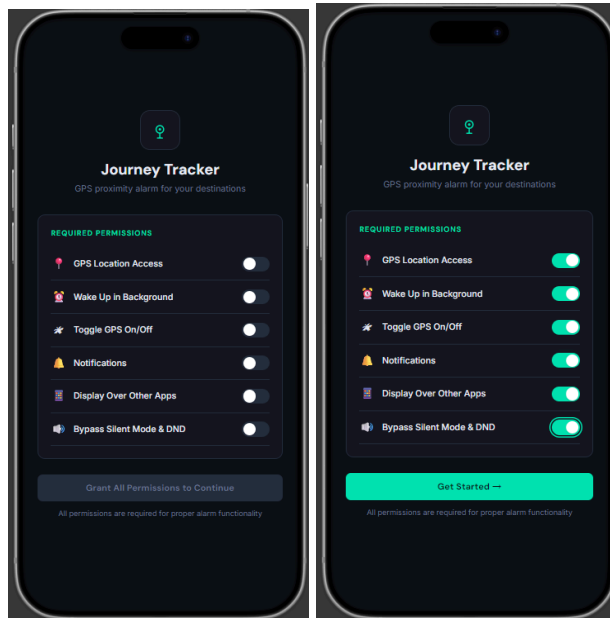
The app uses a local file database for time and current GPS location. During each GPS poll, this database is added with the updated value. This database is then used to estimate: **average travel speed, estimated arrival time, estimated alarm time.**

TODO: Decide on what kind of algorithm to use for the estimation.

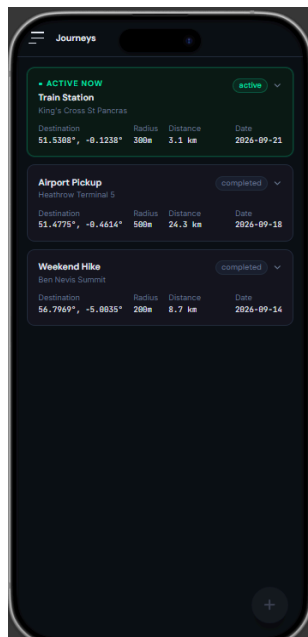
4. UI design

Final Visuals:

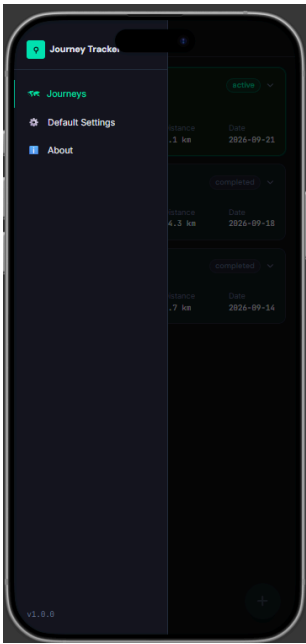
First time opening the app:



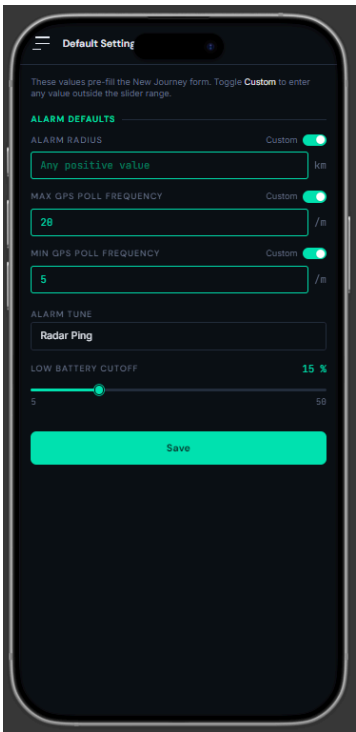
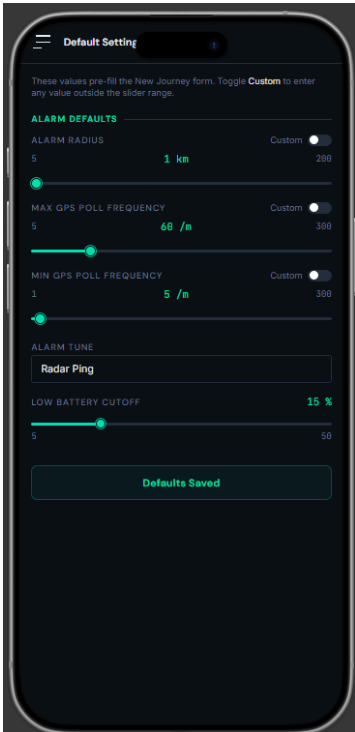
After “Get started”



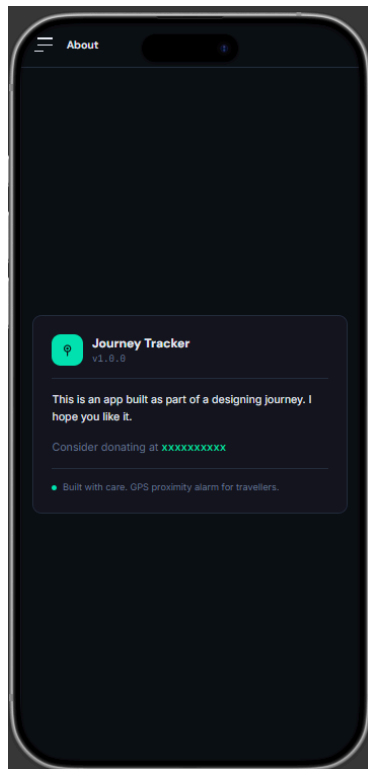
On clicking the hamburger menu:



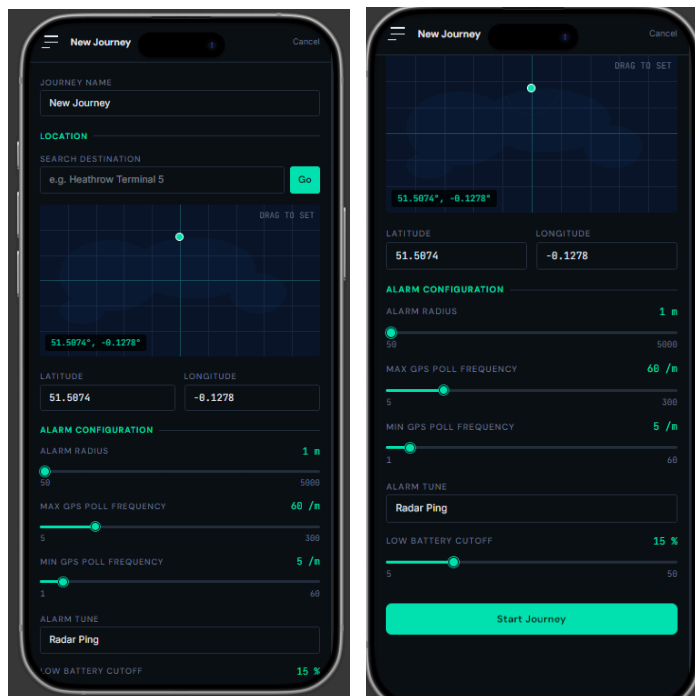
“Default Settings” page:



“About” page:



On clicking the “+” button on “Journeys” page to create a new journey:



(Dont worry about the map here. We have decided to use MapBox anyway)

Figma Make prompt (don't worry much about this. These are the prompts I used on figma make to create the above UI design):

Simple application with 3 screens:

1. Welcome screen for first-time opening the app that prompts user for permissions - (GPS, wake up on background, Permission to turn GPS on/off, Notification, Display over other apps, Bypass silent mode and DND for alarm)
2. Home screen showing cards for past "journeys", with a "+" button in the bottom right to set up a new journey. This pops up a "set up new journey" screen with placeholder values that can be adjusted. There is a hamburger menu, which reveals two other pages: "Default settings", and "about", each of which will take the user to that respective screen..
3. Set up new journey: This screen has a GPS location selection widget that uses a map. The user will be able to both drag a pointer on the map or search for locations. There are also two input fields below for GPS longitude and latitude that are connected to the above map. Other adjustable inputs include: alarm radius, Max GPS polling frequency, Min GPS polling frequency, alarm tune, low battery cutoff. They are pre-filled with placeholder values as set by the user in "Default settings" screen.
4. "Default settings" screen: This screen has the default placeholder values for all the above adjustable input fields
5. "About" screen: Simple text: "This is an app built as part of a designing journey. I hope you like it. Consider donating at xxxxxxxxxx"

=====

This is a field for "frequency". So change the unit to /m (per minute). Remember to include the '/'. Also do the same for min poll frequency

=====

In this default settings screen, add a checkbox/switch that allows the user to enter their custom value in a text box. This way the user can select values that the slider does not allow.

Also alarm radius slider would be between 5 and 200km. User may pick any positive value for this in custom input.

Also make sure max poll frequency is greater than min poll frequency. If the sliders exceed, adjust the otherslider automatically to adjust.

=====

This radius can be a maximum of 40% of the distance between current location and destination. If destination changes to cross this limit, adjust automatically

5. Tech Stack and Architecture

Battery consumption is the most important aspect here. But because this is a GPS-polling app, tech stack does not make a big difference comparatively. It is about the number of GPS polls and background operations.

This will be a fully mobile application for both Android and iOS. Our options are:

- Flutter
- React-Native

Choosing React-Native as it is widely used and must have a bigger community environment. Possibly more options for each component we are going to use.

6. Final stage: Implementation

But first... revisit the above TODOs...

During GPS loss:

Wait for sometime (this time is set by user). If GPS is still not active, raise an alarm for the user to notify no GPS for the set amount of time.

When battery percentage is already below default cutoff:

The journey setup screen's maximum battery cutoff is current battery minus 5%

About Map-GPS component: Now that we have decided on using react-native, we can use one of the map components. Let us use mapbox.

Now to implement everything using Claude code...